

Dossier de conception – v4

Analyse conceptuelle UML et Merise – Système RAG pour hcp.ma

Structure d'accueil : Haut-Commissariat au Plan (HCP), Direction des Systèmes
d'Information Statistiques

Binôme : Ayman El Baida et Saad Mesbah

Période du stage : 1er juillet – fin août 2026

*Révision du dossier de conception initial (`conception_uml_v3.pdf`, 15 juillet) –
mise à jour au 31 août après six semaines d'implémentation et de tests en conditions réelles*

Table des matières

1	Introduction	2
2	Vue d'ensemble de l'architecture	2
3	Diagramme de cas d'utilisation	3
4	Modèle conceptuel de données (MCD)	4
5	Modèle logique de données (MLD)	4
6	Diagramme de classes	5
7	Diagramme de séquence – question chiffrée	6
8	Diagramme de séquence – question conceptuelle	7
9	Diagramme de séquence – question mixte (nouveau)	8
10	Diagramme d'activité – pipeline d'ingestion	8
11	Écarts assumés entre conception et implémentation réelle	9
12	Synthèse	10

1 Introduction

Ce document révisé le dossier de conception initial ([conception_uml_v3.pdf](#), 15 juillet) à la lumière de six semaines d'implémentation et de tests en conditions réelles. L'architecture à haut niveau – séparation stricte entre texte narratif et données chiffrées, routage par le **Routeur** – reste valide et constitue toujours le fil conducteur du système. Trois évolutions substantielles se sont néanmoins ajoutées depuis v3, chacune décidée et testée en conditions réelles :

- une **mémoire conversationnelle** (cache de réponses et historique de session, Redis, 28 juillet) et un module de **reformulation** des questions de suivi (23 août) ;
- un **troisième chemin de traitement**, MIXTE, pour les questions qui demandent à la fois un chiffre et une explication (28 juillet), en plus des deux chemins CHIFFRE/NOTION déjà formalisés en v3 ;
- le remplacement de facto de l'extraction de tableaux PDF/XLSX comme source principale des indicateurs chiffrés par l'**API de la BDS** ([bds.hcp.ma](#), ADR 0004), déjà documenté dans un complément séparé ([complement_conception_bds.pdf](#), 15 juillet) et intégré ici dans une vue d'ensemble unique.

Ce document s'ouvre sur une vue d'ensemble de l'architecture (section 2), puis reprend les huit diagrammes UML/Merise de v3 (cas d'utilisation, MCD, MLD, classes, séquences, activité), auxquels s'ajoute un neuvième diagramme de séquence pour le scénario MIXTE. Chaque diagramme est accompagné d'une analyse qui signale explicitement ce qui a changé depuis v3 – et, tout aussi important, ce qui a été conçu mais jamais implémenté, par souci de fidélité entre ce document et le code réel (voir section 11).

2 Vue d'ensemble de l'architecture

Cette vue d'ensemble précède volontairement les diagrammes UML/Merise détaillés : c'est le même schéma que celui utilisé dans [fiche_cadrage_v4.pdf](#) et [rapport_stage.pdf](#) (fichier [images/architecture.png](#), partagé entre les trois documents), mis à jour ici pour refléter l'état réel du système au 31 août plutôt que la proposition initiale de juillet.

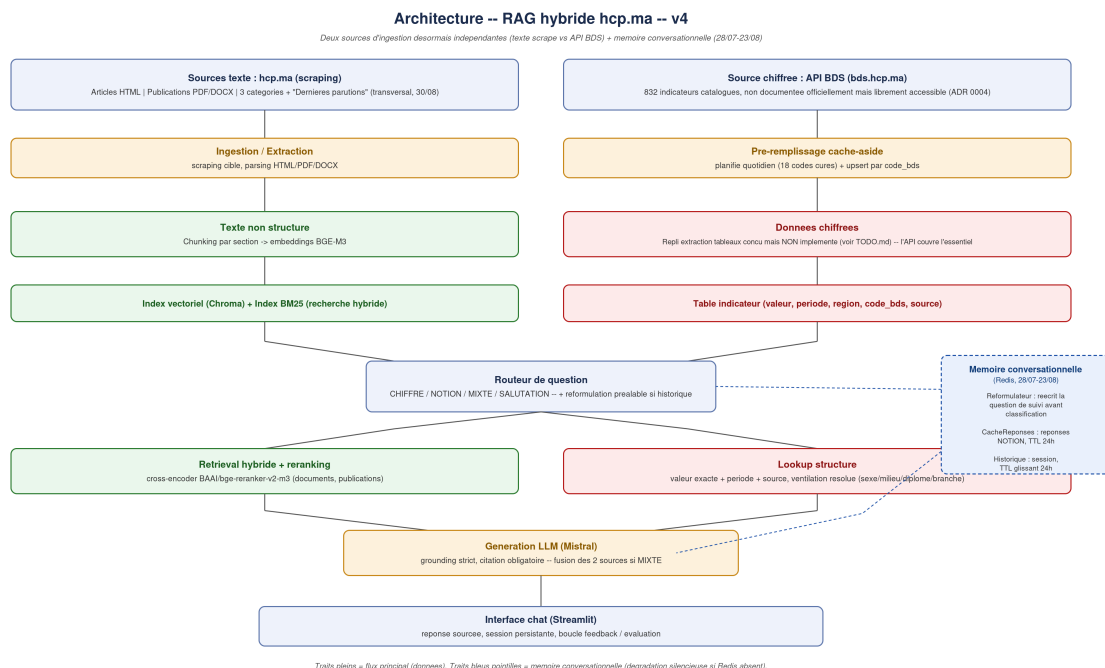


Figure 1 – Architecture générale du système RAG hcp.ma

Analyse

Deux changements distinguent cette version de celle de juillet. D’abord, les **deux sources d’ingestion sont désormais explicitement indépendantes** : la branche texte (scraping hcp.ma, colonne de gauche) et la branche chiffres (API BDS, colonne de droite) ne partagent plus une étape commune d’« Ingestion / Extraction » comme le suggérait le schéma de juillet – elles ont des déclencheurs et des calendriers distincts (voir aussi figure 9). Ensuite, un bloc **mémoire conversationnelle** (Redis, encadré en pointillé bleu à droite) apparaît pour la première fois : il n’existait pas en juillet et s’est ajouté au fil du développement (28 juillet – 23 août). Le reste du schéma – routage, recherche hybride vs lookup structuré, génération LLM avec citation obligatoire, interface de chat – reste fidèle à la proposition initiale.

3 Diagramme de cas d’utilisation

Ce diagramme fixe le périmètre fonctionnel du système en identifiant les acteurs et leurs interactions, avant toute considération technique. **Inchangé depuis v3** : les besoins fonctionnels F1 à F6 (voir [fiche_cadrage_v4.pdf](#)) n’ont pas évolué ; la mémoire conversationnelle et le cas MIXTE sont des raffinements internes du cas d’utilisation « poser une question », pas de nouveaux cas d’utilisation.

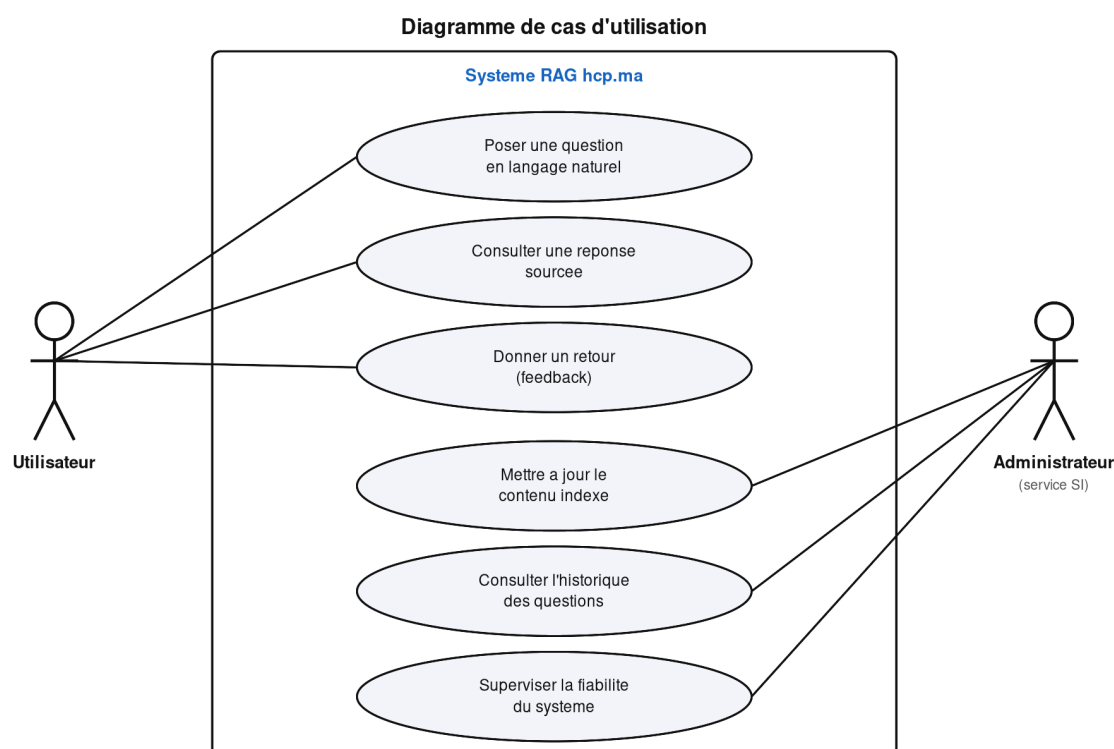


Figure 2 – Diagramme de cas d’utilisation du système RAG hcp.ma

Analyse

Deux acteurs sont identifiés. L’**Utilisateur**, représentant le grand public visiteur de hcp.ma, interagit avec le système pour poser une question en langage naturel, consulter une réponse sourcée, et donner un retour sur cette réponse. L’**Administrateur**, rattaché au service SI, dispose de cas d’utilisation distincts : mettre à jour le contenu indexé, consulter l’historique des questions posées, et superviser la fiabilité du système. Cette séparation des rôles montre que le système doit exposer deux niveaux d’interaction : une interface publique simple et une interface de gestion plus technique.

4 Modèle conceptuel de données (MCD)

Le MCD formalise la partie strictement relationnelle du système, selon la méthode Merise. **Structure inchangée depuis v3** (voir aussi `complement_conception_bds.pdf`, section 2) : aucune entité ni association n’a été ajoutée ou supprimée malgré l’arrivée de l’API BDS – seuls deux attributs évoluent, mis en évidence en rouge sur la figure.

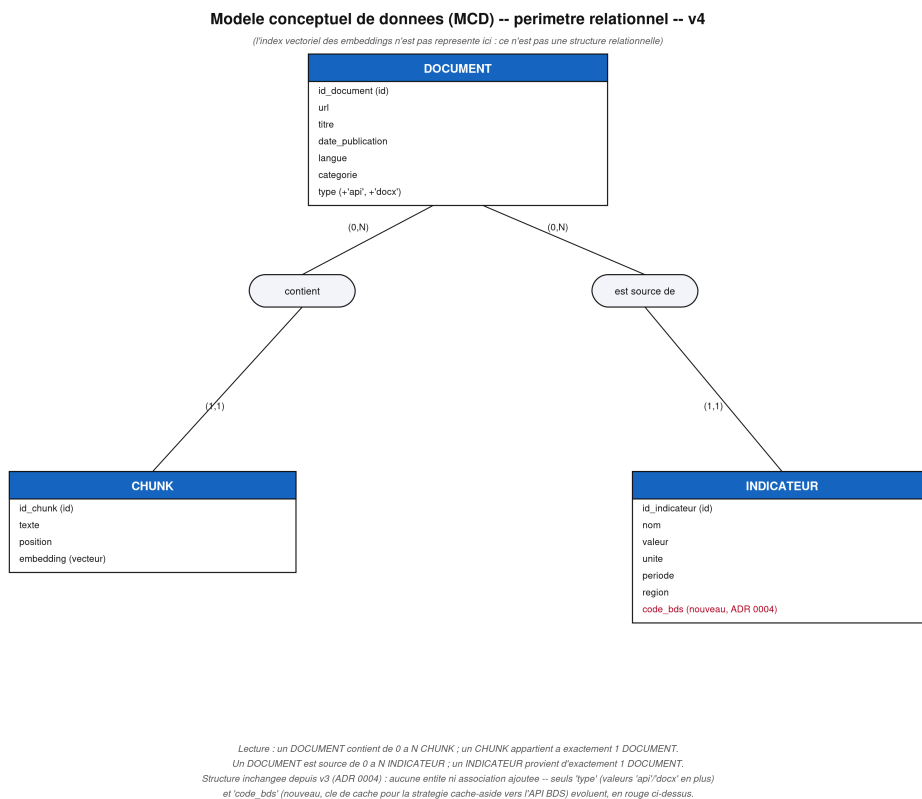


Figure 3 – MCD des entités DOCUMENT, CHUNK et INDICATEUR

Analyse

Trois entités structurent ce modèle : DOCUMENT, CHUNK et INDICATEUR, reliées par les mêmes deux associations qu’en v3 (« contient » et « est source de », cardinalités inchangées). Le domaine de l’attribut `type` de DOCUMENT s’est étendu (`'api'` pour une fiche synthétique représentant un indicateur BDS, `'docx'` pour les notes IPC/IPPI, ADR 0003/0005) et INDICATEUR gagne `code_bds`, clé de la stratégie de cache décrite en section 11. L’index vectoriel des embeddings reste, comme en v3, volontairement hors de ce modèle : ce n’est pas une structure relationnelle.

5 Modèle logique de données (MLD)

Le MLD traduit le MCD en schéma de tables directement implémentable, et correspond exactement à `db/schema.sql` tel qu’exécuté en production.

Analyse

Les deux clés étrangères (`id_document` dans CHUNK et INDICATEUR) sont inchangées depuis v3 et continuent de garantir l’exigence NF3 (traçabilité systématique). La nouveauté est `indicateur.code_bds` (nullable, indexée via `idx_indicateur_code_bds`) : pour une ligne extraite d’un PDF/XLSX, elle reste NULL comme avant ; pour une ligne venant de l’API BDS, elle contient le code stable de l’indicateur (ex. I3181) et sert de clé d’upsert au pré-remplissage nocturne (voir figure 9, colonne B).

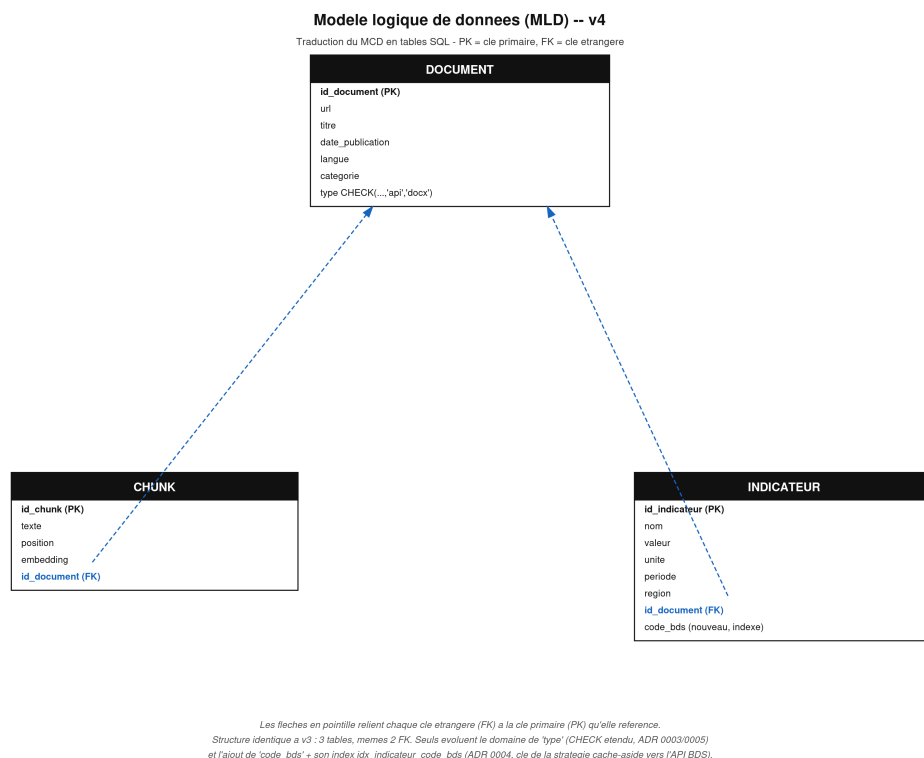


Figure 4 – MLD : tables *DOCUMENT*, *CHUNK* et *INDICATEUR* avec clés primaires et étrangères

6 Diagramme de classes

Ce diagramme transpose les modules du système en classes logicielles. **Évolution la plus visible depuis v3** : deux nouveaux modules transversaux (**Reformulateur**, **CacheRedis**) et un nouveau module d'accès à l'API BDS (**BdsClient**) s'ajoutent aux neuf classes originales, avec une précision importante sur ce qui est réellement câblé (traits pleins) et ce qui a été conçu sans jamais être implémenté (trait gris pointillé – voir section 11).

Analyse

Le cœur du diagramme (Scraper → Extracteur → IndexeurTexte → Routeur → {RetrievalReranker, LookupStructure} → Generateur → Interface) est inchangé depuis v3. Trois ajouts :

- **BdsClient** alimente désormais **ConstructeurIndicateurs** en parallèle de (et en pratique à la place de) l'ancien chemin **Extracteur** → **ConstructeurIndicateurs.structurer(tableaux)**, qui reste dessiné en pointillé gris car jamais implémenté (**NotImplementedError**, priorité basse depuis que l'API BDS couvre l'essentiel des indicateurs curés) ;
- **Reformulateur** s'intercale entre l'arrivée de la question et le **Routeur** : il réécrit une question de suivi ambiguë en question autonome en s'appuyant sur l'historique récent, avant toute classification ;
- **CacheRedis** (regroupant **CacheReponses** et **HistoriqueConversation**) est lu par le **Reformulateur** (historique) et encadre le chemin **NOTION** (cache de réponses) et l'écriture finale de l'historique – en pointillé bleu sur la figure car sa présence est optionnelle : en son absence (pas de serveur Redis), le système continue de fonctionner à l'identique, simplement sans mémoire ni mise en cache.

Le **Routeur.classifier** renvoie désormais quatre valeurs possibles (**CHIFFRE**, **NOTION**, **MIXTE**, **SALUTATION**) au lieu de deux : c'est la source du diagramme de séquence supplémentaire (figure 8).

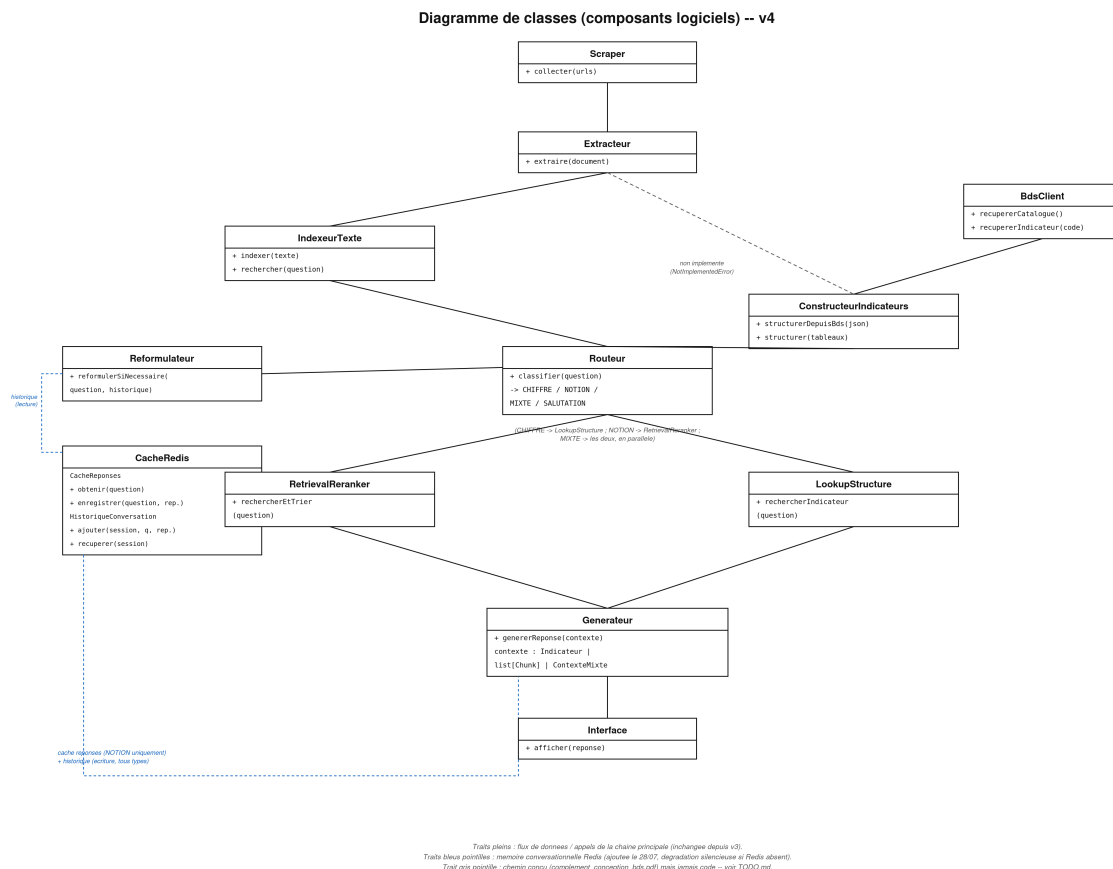


Figure 5 – Diagramme de classes des composants logiciels

7 Diagramme de séquence – question chiffrée

Ce diagramme illustre le chemin de traitement d’une question portant sur une valeur précise. La logique de fond (requête exacte, aucune génération libre sur le chiffre) est inchangée depuis v3 ; les étapes 2-3 et 8 (encadrées dans la figure) sont nouvelles.

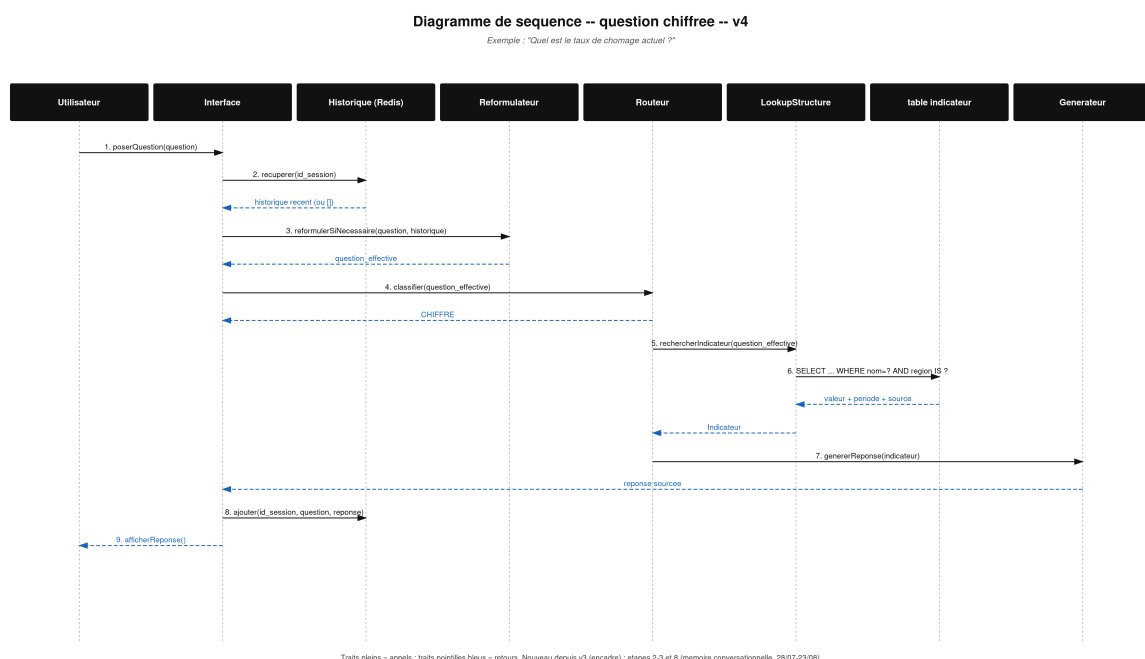


Figure 6 – Scénario « Quel est le taux de chômage actuel ? »

Analyse

La caractéristique essentielle du scénario, identique à v3, est l'absence de génération libre : le **LookupStructure** exécute une requête directe sur la table **indicateur** (alimentée soit par le pré-remplissage BDS, soit – rarement – par extraction PDF/XLSX) et renvoie une valeur exacte accompagnée de sa source. Les étapes nouvelles depuis v3 encadrent ce cœur sans le modifier : la question passe d'abord par **Reformulateur** (qui la laisse inchangée si elle est déjà autonome ou si aucun historique n'existe), et la question **ORIGINALE** (pas la reformulée) est journalisée dans l'historique en toute fin d'échange, pour rester fidèle à ce que l'utilisateur a réellement tapé.

8 Diagramme de séquence – question conceptuelle

Ce diagramme illustre le chemin emprunté pour les questions explicatives ou méthodologiques. **Nouveauté depuis v3** : un point de décision cache (étape 5), qui court-circuite le reranking et la génération LLM lorsque la question normalisée a déjà été répondue dans les dernières 24h.



Figure 7 – Scénario « C'est quoi le RGPH ? »

Analyse

Le chemin « cache miss » (bande du bas) reproduit exactement le scénario de v3 : le **RetrievalReranker** interroge l'index par recherche hybride, retreie les passages, et le **Generateur** synthétise une réponse sourcée. Le chemin « cache hit » (bande du haut, nouveau) court-circuite tout cela : aucun appel au reranker ni au LLM Mistral, la réponse précédemment générée est renvoyée telle quelle. Ce cache ne s'applique volontairement qu'au chemin NOTION pur : le chemin CHIFFRE est déjà une requête SQL directe (rien à gagner à la cacher), et le chemin MIXTE (figure 8) en est délibérément exclu pour que son chiffre reste à jour à chaque appel.

9 Diagramme de séquence – question mixte (nouveau)

Ce diagramme supplémentaire, absent de v3, formalise le troisième chemin de traitement ajouté le 28 juillet : les questions qui demandent à la fois une valeur chiffrée et une explication de son évolution (ex. « pourquoi le chômage a-t-il augmenté ? »).

Diagramme de séquence -- question mixte -- v4 (nouveau, ajouté le 28/07)
Exemple : "Pourquoi le taux de chômage a-t-il augmenté ?" (chiffre + explication)

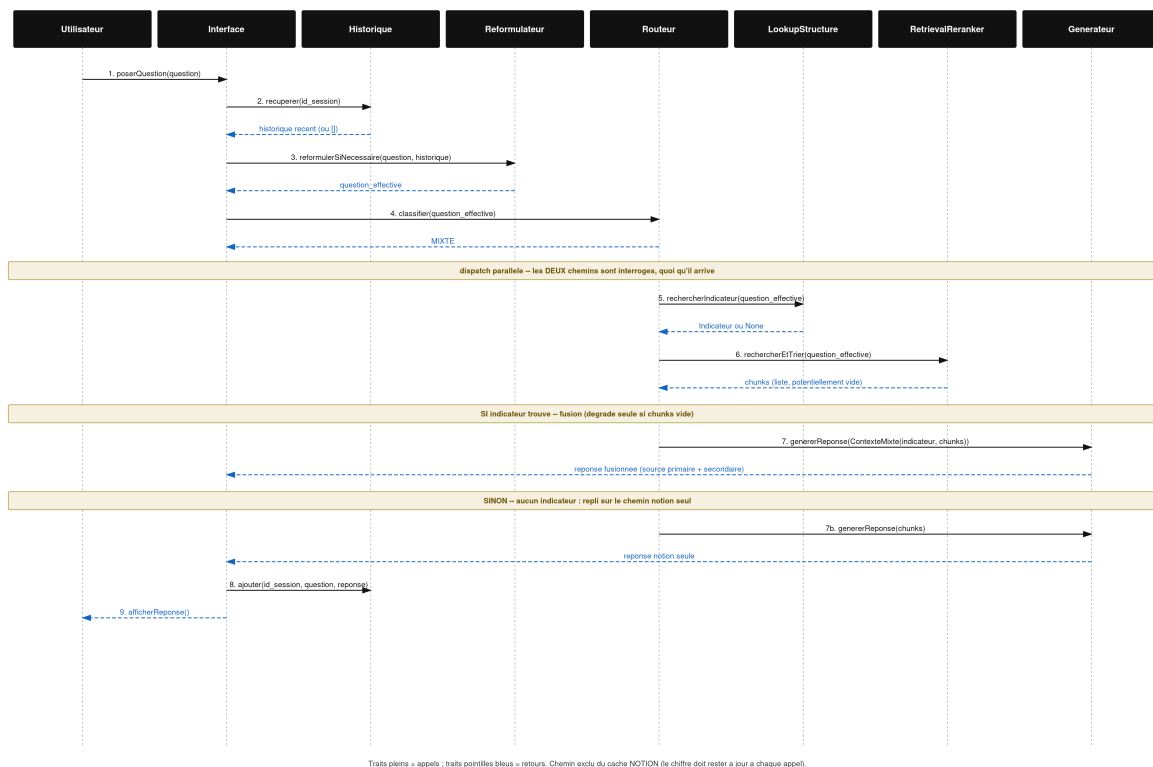


Figure 8 – Scénario « Pourquoi le taux de chômage a-t-il augmenté ? »

Analyse

La caractéristique du scénario MIXTE est le **dispatch parallèle** : contrairement aux deux chemins précédents qui s'excluent mutuellement, **LookupStructure** ET **RetrievalReranker** sont interrogés systématiquement, quoi qu'il arrive. Le **Generateur** dégrade ensuite proprement selon ce qui a été trouvé : chiffre et passages trouvés → réponse fusionnée avec deux sources citées (primaire et secondaire) ; chiffre seul → réponse chiffrée seule ; passages seuls (aucun indicateur trouvé) → repli sur une réponse notion seule. Ce scénario complète la bifurcation CHIFFRE/NOTION déjà présente en v3 sans la remplacer : c'est un troisième chemin, pas une fusion des deux précédents dans le **Routeur** lui-même.

10 Diagramme d'activité – pipeline d'ingestion

Ce diagramme modélise le processus d'ingestion du contenu, en amont de toute interaction avec un utilisateur. **Changement structurel depuis v3** : la bifurcation « type de contenu ? » figurait en v3 comme deux branches d'un même flux forcé/joint. En réalité, six semaines d'implémentation ont montré qu'il s'agit de **deux processus indépendants**, déclenchés différemment (l'un par le scraping, à la demande ou en historique complet ; l'autre par une tâche planifiée quotidienne) – la figure ci-dessous le représente désormais comme tel plutôt que par un faux parallélisme.

Analyse

La colonne A (pipeline documentaire) reprend le chemin texte de v3 à l'identique (découpage → embeddings → indexation hybride), avec une collecte étendue depuis v3 aux quatre flux de découverte

Diagramme d'activité -- pipeline d'ingestion -- v4

Deux processus indépendants (triggers et calendriers distincts), non un seul flux joint

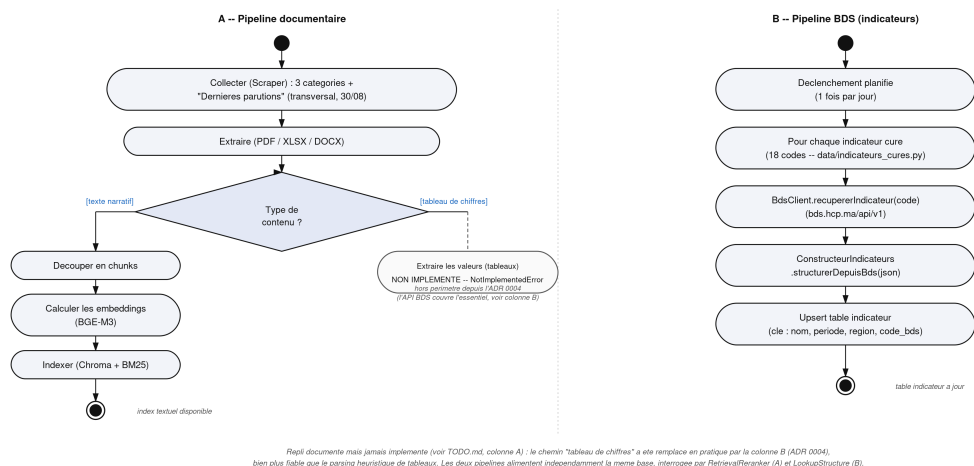


Figure 9 – Pipeline d'ingestion, de la collecte à l'indexation

réels : les trois catégories originales plus le flux transversal « Dernières parutions » (30 août, nécessaire pour des publications comme « Chiffres clés » qui ne relèvent d'aucune des trois catégories). Sa branche « tableau de chiffres », en revanche, reste grise et pointillée : c'est le chemin décrit en v3 mais jamais codé (voir section 11). La colonne B (pipeline BDS, nouveau depuis v3) est le véritable chemin d'alimentation de la table `indicateur` aujourd'hui : un appel planifié quotidien qui met à jour les indicateurs curés par upsert. Les deux colonnes ne se synchronisent pas : elles alimentent la même base indépendamment, chacune interrogée en lecture par un module différent (RetrievalReranker pour A, LookupStructure pour B).

11 Écarts assumés entre conception et implémentation réelle

Par souci de rigueur – et parce que le projet a fait de l'observation en conditions réelles sa discipline centrale (voir JOURNAL.md) – cette section documente explicitement ce qui a été conçu dans les dossiers de conception successifs mais n'a jamais été câblé dans le code, plutôt que de le laisser silencieusement disparaître des diagrammes.

- **ConstructeurIndicateurs.structurer(tableaux)** (repli PDF/XLSX pour les indicateurs, chemin gris pointillé des figures 5 et 9) : la méthode existe dans le code mais lève systématiquement `NotImplementedError`. Déclaré hors périmètre à priorité basse dans `TODO.md` dès que l'API BDS a couvert la majorité des indicateurs ciblés (ADR 0004) – ce n'est pas un bug, c'est un arbitrage de périmètre assumé et tracé.
- **Repli « à la demande » vers l'API BDS** (« Figure A » de `complement_conception_bds.pdf`, section 3) : ce document, rédigé le 15 juillet suite à l'ADR 0004, décrit une cascade à trois niveaux dans `LookupStructure` (cache local → recherche d'un code plausible dans le catalogue → appel direct à l'API avec écriture en retour). Seul le premier niveau (lecture du cache local, alimenté par le pré-remplissage nocturne – colonne B de la figure 9) est réellement implémenté : `src/lookup_structure.py` n'importe jamais `src/bds_client.py`. Le pré-remplissage nocturne (« Figure B » du même document) est, lui, pleinement implémenté et validé à grande échelle (35561 lignes upsertées, 21 juillet – voir `TODO.md`).

Conséquence pratique : si une question chiffrée porte sur un indicateur absent des 18 codes curés pré-remplis, le système ne tente aujourd'hui aucun appel API en direct – il retombe sur le chemin NOTION (recherche textuelle), comme documenté dans le repli « chiffre → notion » déjà présent en v3 (voir figure 6, note de `scripts/poser_question.py`).

12 Synthèse

Ce dossier v4 couvre les mêmes angles qu'en v3 (périmètre fonctionnel, données relationnelles, architecture logicielle, comportement dynamique, préparation des données), précédés désormais d'une vue d'ensemble de l'architecture (figure 1) et complétés par un diagramme supplémentaire pour le troisième chemin de traitement (MIXTE, figure 8) apparu en cours de développement. La séparation stricte entre traitement du texte et traitement des données chiffrées, fil conducteur de v3, demeure intacte ; ce qui a évolué est ce qui alimente chacun des deux côtés (API BDS plutôt qu'extraction de tableaux) et ce qui les entoure désormais (mémoire conversationnelle, reformulation). La section 11 assume explicitement l'écart entre ce qui avait été conçu pour l'intégration BDS et ce qui a effectivement été câblé – un choix de périmètre documenté plutôt qu'une régression cachée. Ce dossier, avec le complément BDS et le journal de bord, constitue la base documentaire du rapport de stage.